

Master Architecture

Document: docs/architecture/MASTER_ARCHITECTURE.md

System: HackNest — Student Operating System

Status: Foundational / Authoritative — resolves Critical finding from ARCHITECTURE_AUDIT_REPORT.md (Section 1, Item 2)

Type: Architecture Specification Only — No Implementation

This document follows docs/architecture/README.md exactly: it occupies the **Governance layer** described there, sits above every subsystem architecture document, and is the document every other document answers to. Where any subsystem document conflicts with this one, this document governs (see "Cross-Document Precedence" below).

1. Vision and System Goals

HackNest is a **Student Operating System** — a single, persistent platform that follows a student through their academic and early-builder journey, unifying identity, academic affiliation, skills, team formation, project work, hackathon participation, and community presence behind one account.

System goals:

1. **Unify fragmented student tooling.** Replace the scattered combination of resume sites, Discord servers, hackathon platforms, and portfolio tools with one coherent system.
 2. **Be trustworthy with student data.** Every architectural decision defers to security and privacy as a precondition, not a feature (see Section 13).
 3. **Scale to the full addressable student population.** Architected from the outset for at least one million students (see Section 12), not retrofitted for scale later.
 4. **Stay legible as the system grows.** A large number of domains (25+, per DATABASE_ARCHITECTURE.md) must not become a large number of inconsistent patterns. Consistency is enforced structurally, through this document, not left to convention alone.
 5. **Make "done" checkable.** Every subsystem document defines standards precise enough that conformance can be validated, not just asserted.
-

2. Core Architecture Principles

These principles govern every subsystem document and every implementation phase. Where a subsystem document states its own principles (e.g., `SECURITY_ARCHITECTURE.md`'s Zero Trust, `UI_DESIGN_SYSTEM.md`'s Mobile-first), those are refinements of the principles below, applied to that subsystem — never a replacement of them.

- **Server-first.** The server is the source of truth for data access, business rules, and authorization. Clients render and collect input; they do not decide what is true or what is allowed.
- **Domain-owned, not feature-owned.** The system is organized around a fixed set of domains (Authentication, Students, Teams, Projects, Hackathons, etc. — see `DATABASE_ARCHITECTURE.md`), and every feature extends an existing domain's contract rather than inventing a parallel structure.
- **Defense in depth.** No single layer (database RLS, API authorization, frontend validation) is trusted alone to enforce a security or correctness guarantee. Each layer enforces independently.
- **Specify before building.** No domain, API surface, UI pattern, or infrastructure component is implemented before it is specified in the relevant subsystem document. This principle is the direct governing response to the root cause identified in `ARCHITECTURE_AUDIT_REPORT.md` — documents and implementation proceeding ahead of an authoritative specification.
- **One authoritative owner per concern.** Every specification concern (e.g., "what does an audit log record contain," "what are the rate-limit tiers") has exactly one document that owns its definition. Other documents may describe how they interact with that concern, but never redefine it. See Section 9.
- **Everything is versioned and changeable, nothing is silently changed.** See Section 10 and Section 11.

3. Layered Architecture

HackNest's architecture — both the documentation and the system it describes — is organized into four layers:

1. **Governance layer.** `README.md`, `MASTER_ARCHITECTURE.md` (this document), `MASTER_GOVERNANCE_AMENDMENT_001.md`. Defines what the system is, how it's organized, and how decisions about its architecture are made and changed.
2. **Subsystem architecture layer.** `DATABASE_ARCHITECTURE.md`, `API_ARCHITECTURE.md`, `SECURITY_ARCHITECTURE.md`, `UI_DESIGN_SYSTEM.md`, `TESTING_STRATEGY.md`, `DEPLOYMENT_ARCHITECTURE.md`,

`OPERATIONS_ARCHITECTURE.md` . Each owns one concern in full depth, in service of the goals and principles defined here.

3. **Execution layer.** `IMPLEMENTATION_ROADMAP.md` , `PHASE_TEMPLATE.md` . Sequences actual construction against the two layers above.

4. **Verification layer.** `ARCHITECTURE_AUDIT_REPORT.md` (and any future audits). Checks that the layers above remain internally consistent.

In the running system itself (not the documentation), the same layering principle holds at the application level: Client → Next.js Application (Server Components / Server Actions / Route Handlers) → Supabase (Auth, Database with RLS, Storage) → underlying infrastructure, exactly as described in `DEPLOYMENT_ARCHITECTURE.md`'s System Architecture section. No layer is skipped (e.g., the client never talks to Supabase directly for authenticated data).

4. Module Boundaries

A **module** in HackNest corresponds to a **domain**, as enumerated authoritatively in `DATABASE_ARCHITECTURE.md` (Authentication, Students, Universities, Profiles, Skills, Teams, Projects, Hackathons, Organizations, Communities, Chat, Messages, Notifications, Search, Feed, Achievements, Bookmarks, Files, Admin, Analytics, Audit Logs, Settings, and any domain added under the change process in Section 11).

Boundary rules:

- A module owns its own data shape, its own API contract surface, and its own UI patterns where applicable — but the *specification* of each of these still lives in the relevant subsystem document (data shape in `DATABASE_ARCHITECTURE.md` , API contract in `API_ARCHITECTURE.md` , UI patterns in `UI_DESIGN_SYSTEM.md`), not scattered ad hoc.
 - A module may depend on another module's public contract (e.g., Teams depends on Students existing) but never reaches into another module's internal implementation.
 - A module that needs data or behavior from another module requests it through that module's defined API contract — modules do not share direct database access to each other's tables outside of what RLS and the API layer already expose.
 - No module is permitted to duplicate another module's responsibility. If two modules appear to need the same capability (e.g., both Chat and Notifications need to know about new messages), one module owns the source of truth and the other consumes it — this is the same principle used to resolve the audit-log ownership ambiguity (Section 9).
-

5. Ownership Rules

- **Document ownership.** Each subsystem document is the sole authoritative source for its concern (see Section 9's table). Any other document referencing that concern must cite the owning document rather than restate its content.
 - **Domain ownership.** Each domain listed in Section 4 has exactly one owning module. Cross-domain features (e.g., Feed, which surfaces events from Teams/Projects/Hackathons) are themselves domains with their own ownership boundary — they own *composition*, not the underlying source domains' data.
 - **Role ownership.** The canonical role set is: **student, moderator, admin, service** (as established across `SECURITY_ARCHITECTURE.md`, `TESTING_STRATEGY.md`, and `IMPLEMENTATION_ROADMAP.md`). This document confirms and freezes that set as authoritative. Any new role must be added through the change process in Section 11, and must be reflected consistently across every subsystem document that references roles — not redefined locally by any one of them.
 - **Human ownership (interim).** Until `MASTER_GOVERNANCE_AMENDMENT_001.md` exists, approval authority for architecture changes rests with whoever holds final review authority over this document's repository — this document does not invent a governance body it cannot verify exists. This is a deliberate, flagged placeholder (see Section 16, Assumptions).
-

6. Dependency Rules

- **Acyclic by construction.** No domain, module, or subsystem document may depend on something that depends on it, directly or transitively. `ARCHITECTURE_AUDIT_REPORT.md` identified two violations of this rule at the implementation-phase level (Phase 009 ↔ 010, Phase 014 ↔ 015); this document establishes the rule those phases violated, so that `IMPLEMENTATION_ROADMAP.md` can be corrected against it in a future remediation pass (the Roadmap itself is out of scope for this document to edit, per the current instruction to produce specification documents only).
- **Dependency direction follows the layers in Section 3.** Execution-layer documents depend on subsystem-layer documents, which depend on governance-layer documents. The reverse is never true — this document, for example, never depends on `IMPLEMENTATION_ROADMAP.md` for its own correctness.
- **A module's dependencies must be declared, not incidental.** If a feature in one domain needs another domain's data, that dependency is stated explicitly in the owning subsystem document (e.g., `DATABASE_ARCHITECTURE.md`'s relationship definitions), not discovered implicitly during implementation.

- **Optional vs. hard dependencies are distinguished.** A module may have a hard dependency (cannot function at all without the other) or a soft/enrichment dependency (functions in a reduced form without the other). Documents and roadmap phases must state which kind applies — ambiguity here was a contributing cause of the circular dependencies identified in the audit.

7. Naming Conventions

Pending the full `CODING_STANDARDS.md`, this document establishes the naming conventions that govern cross-document consistency (code-level naming detail, such as file/module casing, remains owned by `CODING_STANDARDS.md` once produced):

- **Domains** are named as singular-concept, plural-entity nouns matching the canonical list in Section 4 (e.g., "Teams," not "TeamManagement" or "TeamService").
- **Roles** are named exactly as listed in Section 5: `student`, `moderator`, `admin`, `service` — lowercase in system contexts, capitalized only in prose.
- **Documents** are named `SCREAMING_SNAKE_CASE.md` inside `docs/architecture/`, matching the pattern already established by every existing document.
- **Cross-document references** always use the exact filename in backticks (e.g., ``DATABASE_ARCHITECTURE.md``) rather than a paraphrased title, so references remain unambiguous and greppable.

8. Folder Organization

At the architecture-documentation level (the only level this document governs directly):

```
docs/  
  architecture/  
    README.md  
    MASTER_ARCHITECTURE.md  
    MASTER_GOVERNANCE_AMENDMENT_001.md  
    PHASE_TEMPLATE.md  
    CODING_STANDARDS.md  
    DATABASE_ARCHITECTURE.md  
    API_ARCHITECTURE.md  
    SECURITY_ARCHITECTURE.md  
    UI_DESIGN_SYSTEM.md  
    TESTING_STRATEGY.md  
    DEPLOYMENT_ARCHITECTURE.md  
    OPERATIONS_ARCHITECTURE.md
```

IMPLEMENTATION_ROADMAP.md
ARCHITECTURE_AUDIT_REPORT.md

Application-level folder organization (source code structure, e.g., where Server Actions or components live) is explicitly **not** defined by this document — that belongs to CODING_STANDARDS.md, once produced, so that this document does not encroach on that document's ownership (per Section 9's exclusivity rule).

9. Cross-Document Precedence

This section is the direct governance mechanism for resolving the "duplicate/overlapping specifications" findings in ARCHITECTURE_AUDIT_REPORT.md (Section 6).

Precedence order, highest to lowest:

- MASTER_ARCHITECTURE.md (this document)
- MASTER_GOVERNANCE_AMENDMENT_001.md (once it exists — governs process, not technical content, so it takes precedence on process questions only)
- Subsystem architecture documents, each authoritative **within its own owned concern only** (table below)
- IMPLEMENTATION_ROADMAP.md
- Any other document

Exclusive concern ownership (resolving the audit's overlap findings):

Concern	Sole Owning Document
Domain data shape, relationships, RLS strategy, storage, migrations	DATABASE_ARCHITECTURE.md
Audit log record specification (what fields, what triggers a record, retention)	DATABASE_ARCHITECTURE.md — resolving the three-way ambiguity the audit identified between it, SECURITY_ARCHITECTURE.md, and API_ARCHITECTURE.md. Those two documents may describe <i>behavior around</i> audit logs (e.g., "sensitive actions are logged") but must cite DATABASE_ARCHITECTURE.md for the record specification itself rather than restating it.
Request/response contracts, versioning, rate-limit tier enumeration	API_ARCHITECTURE.md — including the actual numeric/tiered rate-limit definitions the audit found were only ever referenced, never defined.

Concern	Sole Owning Document
	<code>API_ARCHITECTURE.md</code> must be updated, when next revised, to actually enumerate its tiers rather than deferring them further.
Authentication/authorization architecture, secrets, compliance	<code>SECURITY_ARCHITECTURE.md</code>
Visual system, components, accessibility standards	<code>UI_DESIGN_SYSTEM.md</code>
Testing levels, quality gates, bug severity	<code>TESTING_STRATEGY.md</code>
Infrastructure, environments, CI/CD, release process	<code>DEPLOYMENT_ARCHITECTURE.md</code>
Day-to-day operational ownership, on-call, incident handling in practice	<code>OPERATIONS_ARCHITECTURE.md</code>

Numeric placeholder ownership (resolving the audit's duplicated-placeholder

finding): Recovery Point/Time Objectives are owned by

`DEPLOYMENT_ARCHITECTURE.md` alone (not duplicated into

`SECURITY_ARCHITECTURE.md`, which should cite it instead on future revision). WCAG

conformance level is owned by `UI_DESIGN_SYSTEM.md` alone (not duplicated into

`TESTING_STRATEGY.md`, which should cite it instead on future revision).

10. Versioning Strategy

- Every document in `docs/architecture/` carries an implicit version equal to its last-modified state; material changes to a document's authoritative content are treated as a new version of that document, not a silent edit.
- This document (`MASTER_ARCHITECTURE.md`) is versioned independently of the application's own release versioning (defined in `DEPLOYMENT_ARCHITECTURE.md`'s Release Strategy) — architecture documentation and shipped software version on different clocks, since architecture can change without a release and vice versa.
- A subsystem document that changes in a way that affects another document's assumptions (e.g., `DATABASE_ARCHITECTURE.md` adding a domain) obligates a review of every document that referenced the old state, per the Change Management process below.

11. Change Management

Until `MASTER_GOVERNANCE_AMENDMENT_001.md` formalizes this, the following interim process governs changes to any document in `docs/architecture/`:

1. A proposed change is written against the specific document(s) it affects.
2. The change is checked against this document's Cross-Document Precedence (Section 9) to confirm it doesn't redefine a concern owned elsewhere.
3. Every document that cites the changed concern is identified and reviewed for consistency (a lightweight version of the audit process in `ARCHITECTURE_AUDIT_REPORT.md`).
4. The change is approved by the interim ownership defined in Section 5 and merged.
5. If the change adds a new domain, role, or subsystem document, this document (`MASTER_ARCHITECTURE.md`) is updated to reflect it, since Sections 4 and 5 are meant to be the canonical, current list.

This process is intentionally lightweight and will be superseded, not duplicated, once `MASTER_GOVERNANCE_AMENDMENT_001.md` exists.

12. Non-Functional Requirements

- **Availability.** Production availability targets are owned by `DEPLOYMENT_ARCHITECTURE.md`; this document only asserts that availability is a first-class, explicitly tracked requirement, not an assumed byproduct of correctness.
 - **Consistency.** User-facing data (e.g., team membership, project status) must be strongly consistent for the acting user's own writes; eventual consistency is acceptable only for cross-domain aggregation surfaces (Feed, Search, Analytics), which must clearly communicate their own staleness where relevant.
 - **Maintainability.** A domain's specification, contract, and implementation must remain traceable to each other — no domain should exist in code without a corresponding entry in `DATABASE_ARCHITECTURE.md`'s domain list.
 - **Portability.** While the current stack targets Vercel/Supabase/Next.js specifically (per `DEPLOYMENT_ARCHITECTURE.md`), domain and API specifications are written to be conceptually portable — they describe responsibilities and contracts, not vendor-specific mechanics, so a future infrastructure change would not require rewriting the subsystem architecture documents themselves.
-

13. Security Principles

Full security architecture is owned by `SECURITY_ARCHITECTURE.md`. This document asserts the following as governance-level, non-negotiable constraints that no subsystem document or roadmap phase may weaken:

- Security review is mandatory, not optional, for any change touching authentication, authorization, or sensitive student data (as already established in `SECURITY_ARCHITECTURE.md` and reflected in `IMPLEMENTATION_ROADMAP.md`).
 - No subsystem document may introduce a data-access pattern that bypasses RLS and API-layer authorization operating independently (defense in depth, per Section 2).
 - Privacy by Design (data minimization, purpose limitation) governs every new domain added under Section 11's change process — a new domain proposal must state what data it collects and why before it is approved.
-

14. Performance Goals

Full performance architecture is owned by `TESTING_STRATEGY.md` (targets and validation) and `DEPLOYMENT_ARCHITECTURE.md` (scaling mechanics). This document asserts the governance-level requirement that:

- Every domain must have defined response-time expectations before it is considered production-ready (closing the gap the audit identified where numeric targets were repeatedly deferred without ever being assigned an owner). `TESTING_STRATEGY.md` is designated as that owner per Section 9, and must be revised to actually assign numeric targets rather than deferring them indefinitely.
 - Performance is validated at the scale defined in Section 15, not only at low, seeded test-data volume.
-

15. Scalability Principles

- HackNest is architected to support **at least one million students**, consistent with the target already established in `DATABASE_ARCHITECTURE.md`'s intended scope and `TESTING_STRATEGY.md`'s Scalability Testing section.
- Scalability is a property of the data layer (indexing, partitioning readiness), the application layer (stateless horizontal scaling), and the delivery layer (CDN caching) working together — no single layer is expected to absorb scale on behalf of the others, consistent with the layered System Architecture in `DEPLOYMENT_ARCHITECTURE.md`.

- Scalability validation is a required, not optional, gate before production readiness, per the Release Quality Gates owned by `TESTING_STRATEGY.md`.
-

16. Documentation Standards

- Every subsystem document must state, at minimum: its purpose, the principles it operates under, the concern(s) it exclusively owns per Section 9, and a validation section confirming internal consistency and consistency with this document.
 - Every subsystem document must cross-reference this document (`MASTER_ARCHITECTURE.md`) as its governing authority, and must cross-reference `README.md` as the index it belongs to.
 - No subsystem document may restate another document's owned concern in detail; it must cite it instead (per Section 9).
 - Assumptions made in the absence of a not-yet-produced document must be explicitly flagged (as every document in this workstream has done), and must be revisited once the relevant document exists.
-

Definition of Done

- [x] Architecture only — no implementation code, SQL, API endpoints, or UI code.
 - [x] Follows `README.md`'s layering model exactly.
 - [x] All required sections present (Vision, Principles, Layered Architecture, Module Boundaries, Ownership Rules, Dependency Rules, Naming Conventions, Folder Organization, Cross-Document Precedence, Versioning, Change Management, NFRs, Scalability, Security, Performance, Documentation Standards).
 - [x] Resolves applicable audit findings (see Validation Report below).
-

Validation Report

Internal Consistency

This document defines a single, non-contradictory hierarchy: Governance → Subsystem → Execution → Verification (Section 3), a single canonical domain list source (Section 4, deferring enumeration to `DATABASE_ARCHITECTURE.md`), a single canonical role set (Section 5), and a single precedence and ownership table (Section 9) that every other section defers to rather than re-deciding. No section of this document contradicts another.

Consistency with README.md

This document occupies exactly the "Governance layer" position README.md described for it, and does not redefine README.md's own content (the document index) — it only references it. Confirmed consistent.

Audit Findings Resolved by This Document

Audit Finding	Resolution
Section 1, Item 2 — Missing MASTER_ARCHITECTURE.md (Critical)	Resolved: this document now exists and is authoritative.
Section 6 — Audit logging owned by 3 documents	Resolved in principle: Section 9 assigns sole ownership to DATABASE_ARCHITECTURE.md. Full resolution requires that document (next in sequence) to actually contain the specification, and requires a future revision of SECURITY_ARCHITECTURE.md / API_ARCHITECTURE.md to cite rather than restate it.
Section 6 — RPO/RTO duplicated across 2 documents	Resolved in principle: Section 9 assigns sole ownership to DEPLOYMENT_ARCHITECTURE.md. Full resolution requires a future revision pass on SECURITY_ARCHITECTURE.md.
Section 6 — WCAG level duplicated across 2 documents	Resolved in principle: Section 9 assigns sole ownership to UI_DESIGN_SYSTEM.md. Full resolution requires a future revision pass on TESTING_STRATEGY.md.
Section 6 — Rate-limit tiers referenced but never enumerated	Resolved in principle: Section 9 confirms API_ARCHITECTURE.md as sole owner and flags that it must be revised to actually enumerate tiers.
Section 6 — Role/domain terminology drift risk	Resolved: Section 4 (domains) and Section 5 (roles) now freeze a single canonical list that every subsystem document must conform to going forward.
Section 5/7 — Circular dependencies in IMPLEMENTATION_ROADMAP.md (Phase 009 ↔ 010, 014 ↔ 015)	Not resolved by this document — this document establishes the acyclic dependency rule (Section 6) that those phases violate, but correcting the roadmap itself is out of scope for a specification-only document per the current instruction. Flagged for a future roadmap revision pass.

Outstanding Dependencies

This document does not yet have `MASTER_GOVERNANCE_AMENDMENT_001.md`, `PHASE_TEMPLATE.md`, or `CODING_STANDARDS.md` to fully govern against — Sections 5, 7, 8, and 11 each contain explicit interim placeholders pending those documents, consistent with this workstream's established practice of flagging rather than silently assuming.

Status: Validated — Internally Consistent — Ready to Govern Remaining Architecture Documents